Singleton

Creational Design Pattern

Provide one shared instance through a controlled access point.

CASE STUDY: Shared database connection

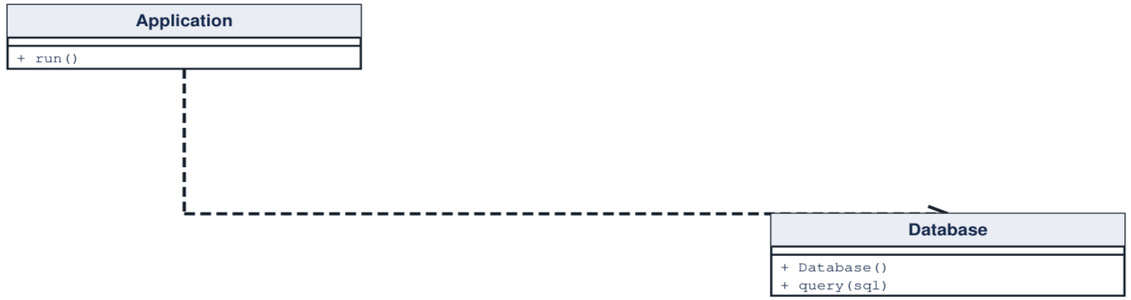
Problem

Independent callers create separate database connections.

Solution

Hide construction; return the same instance to every caller.

Before (direct coupling)



After (using Singleton)



Client Code (Before)

Pseudocode

```
a = new Database()
b = new Database()
a.query(sql)
// a and b are different objects
```

Client Code (After)

Pseudocode

```
a = Database.getInstance()
b = Database.getInstance()
a.query(sql)
assert a is b
```

Key Implementation / Notable Module

Pseudocode

```
// Database: lazy, thread-safe accessor
static synchronized getInstance():
    if instance == null: instance = new Database()
    return instance
```

Source: https://refactoring.guru/design-patterns/singleton

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Factory Method

Creational Design Pattern

Let creator subclasses choose the product used by shared business logic.

CASE STUDY: Road and sea logistics

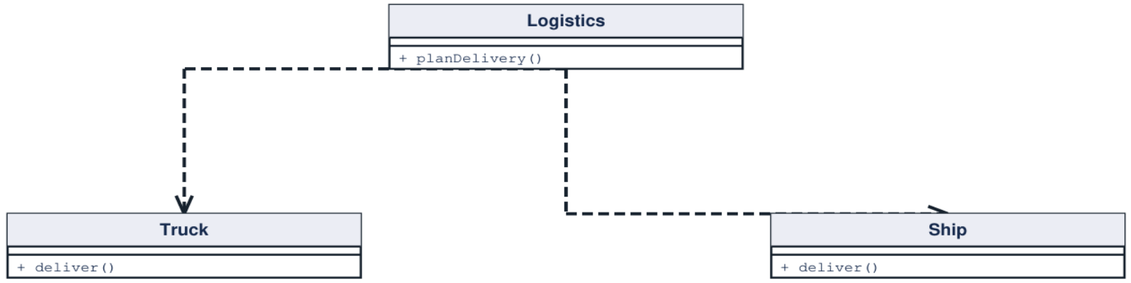
Problem

Logistics constructs concrete transports; each new kind forces edits.

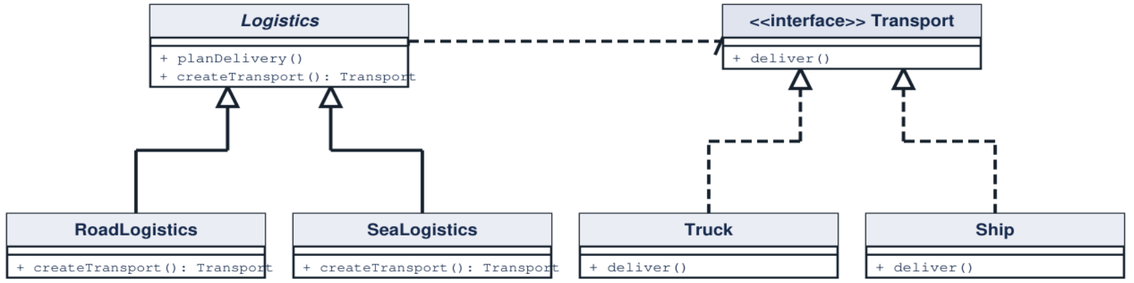
Solution

Override a creation method while preserving delivery planning.

Before (direct coupling)



After (using Factory Method)



Client Code (Before)

Pseudocode

```
truck = new Truck()
ship = new Ship()
truck.deliver()
// every new kind edits Logistics
```

Client Code (After)

Pseudocode

```
logistics = new SeaLogistics()
logistics.planDelivery()
// planning uses Transport
```

Key Implementation / Notable Module

Pseudocode

```
// Logistics.planDelivery()
vehicle = createTransport()
vehicle.deliver()
// SeaLogistics.createTransport(): return new Ship()
```

Source: https://refactoring.guru/design-patterns/factory-method

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Abstract Factory

Creational Design Pattern

Create related products through a family-specific factory.

CASE STUDY: Matching furniture families

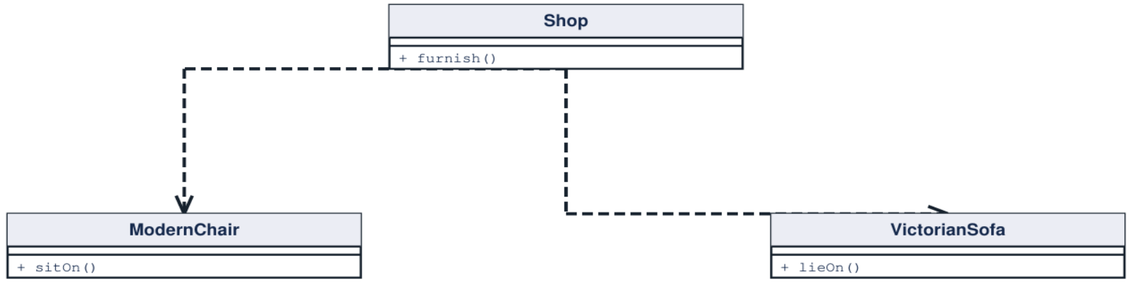
Problem

Direct construction can mix modern chairs with Victorian sofas.

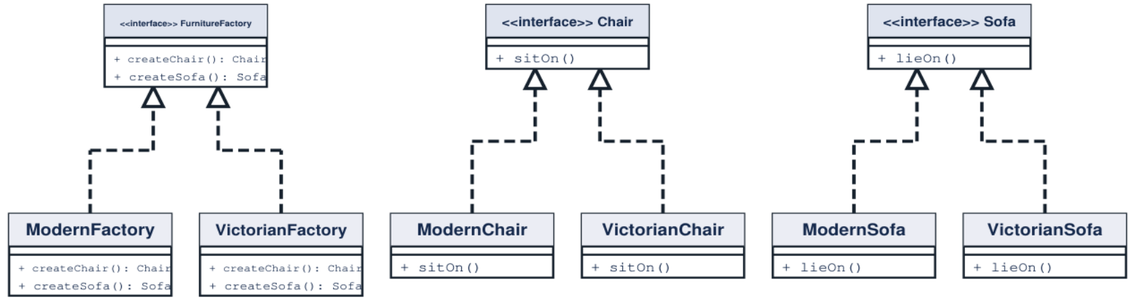
Solution

Select one family factory; request products through interfaces.

Before (direct coupling)



After (using Abstract Factory)



Client Code (Before)

Pseudocode

```
chair = new ModernChair()
sofa = new VictorianSofa()
// mismatched furniture family
```

Client Code (After)

Pseudocode

```
factory = new ModernFactory()
chair = factory.createChair()
sofa = factory.createSofa()
```

Key Implementation / Notable Module

Pseudocode

```
// ModernFactory
createChair(): return new ModernChair()
createSofa():  return new ModernSofa()
// VictorianFactory returns Victorian products
```

Source: https://refactoring.guru/design-patterns/abstract-factory

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Builder

Creational Design Pattern

Separate construction steps from the representation being assembled.

CASE STUDY: Cars and their manuals

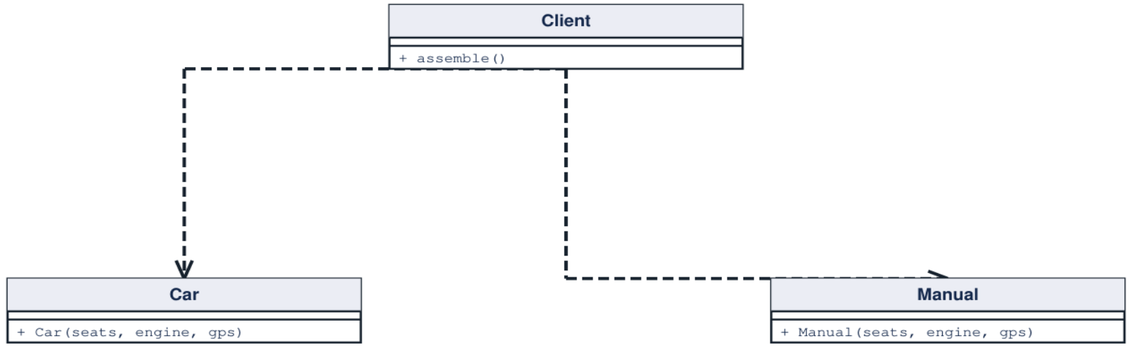
Problem

Long car constructors and separate manual assembly duplicate choices.

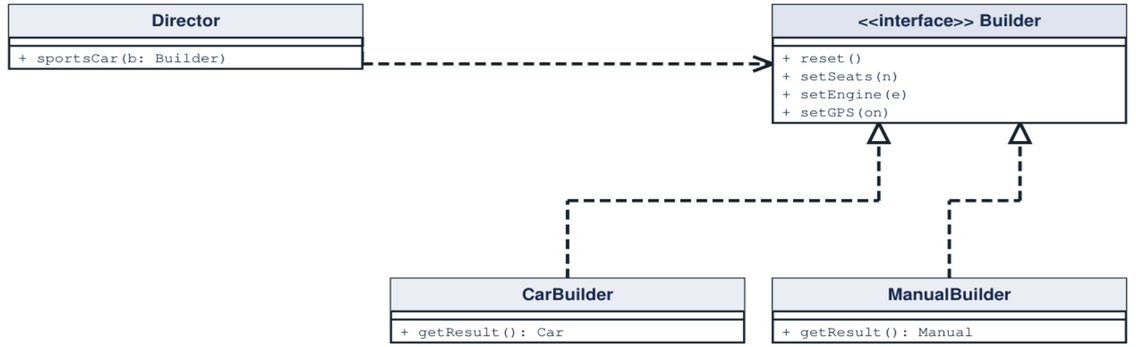
Solution

A director runs the same recipe against different builders.

Before (direct coupling)



After (using Builder)



Client Code (Before)

Pseudocode

```
car = new Car(2, sportEngine, true)
manual = new Manual(2, "sport", "GPS")
// maintain the same recipe twice
```

Client Code (After)

Pseudocode

```
builder = new CarBuilder()
director.sportsCar(builder)
car = builder.getResult()
// use ManualBuilder for a matching manual
```

Key Implementation / Notable Module

Pseudocode

```
// Director.sportsCar(builder)
builder.reset()
builder.setSeats(2)
builder.setEngine(sportEngine)
builder.setGPS(true)
```

Source: https://refactoring.guru/design-patterns/builder

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Adapter

Structural Design Pattern

Translate an existing interface into the interface a client expects.

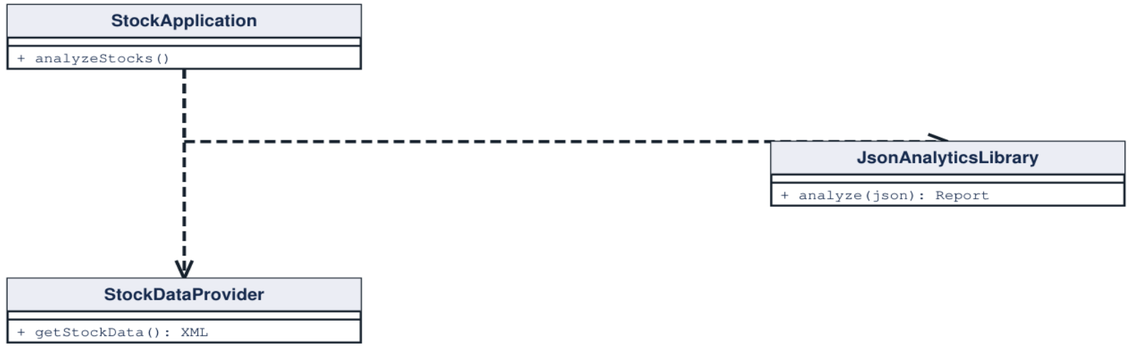
Problem

The stock app supplies XML; the analytics library requires JSON.

Solution

An adapter accepts XML, converts it, then delegates analytics.

Before (direct coupling)



After (using Adapter)



Client Code (Before)

Pseudocode

```
xml = provider.getStockData()
library = new JsonAnalyticsLibrary()
report = library.analyze(xml)
// incompatible input format
```

Client Code (After)

Pseudocode

```
service = new XmlAnalyticsAdapter(library)
xml = provider.getStockData()
report = service.analyze(xml)
```

Key Implementation / Notable Module

Pseudocode

```
// XmlAnalyticsAdapter.analyze(xml)
json = convertXmlToJson(xml)
return library.analyze(json)
```

Source: https://refactoring.guru/design-patterns/adapter

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Bridge

Structural Design Pattern

Separate two independent dimensions so both can evolve independently.

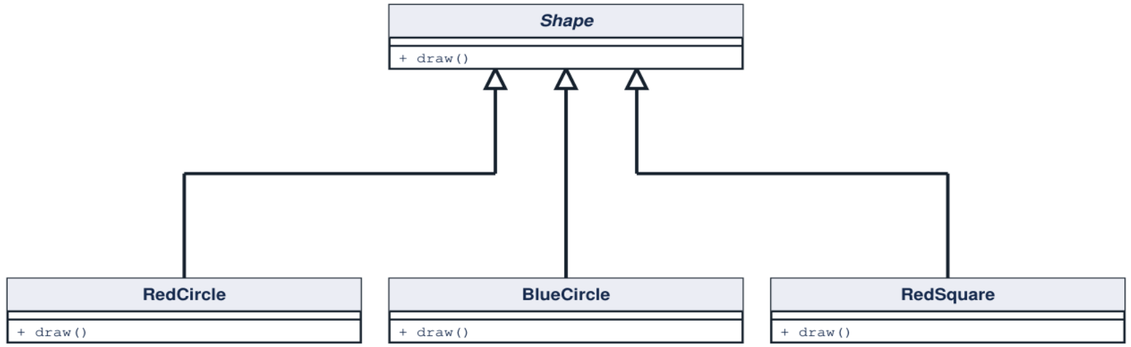
Problem

Each shape/color combination requires another subclass.

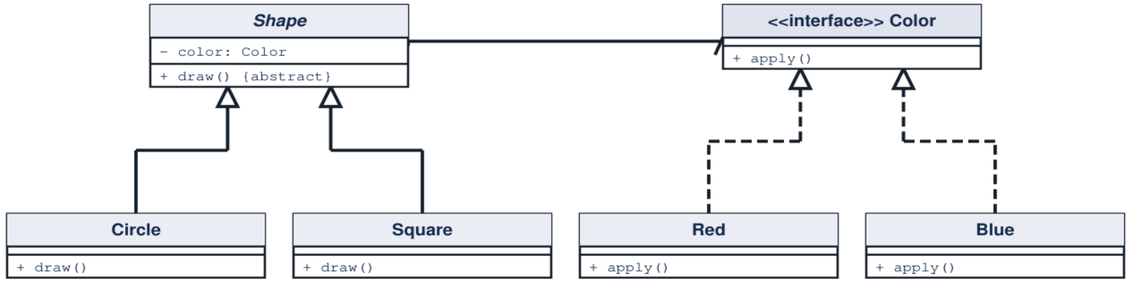
Solution

Shapes hold a Color reference instead of encoding color in inheritance.

Before (direct coupling)



After (using Bridge)



Client Code (Before)

Pseudocode

```
redCircle = new RedCircle()
blueSquare = new BlueSquare()
redCircle.draw()
// more combinations, more classes
```

Client Code (After)

Pseudocode

```
shape = new Circle(new Red())
shape.draw()
shape.color = new Blue()
shape.draw()
```

Key Implementation / Notable Module

Pseudocode

```
// Circle.draw()
color.apply()
renderCircleGeometry()
// color and geometry vary independently
```

Source: https://refactoring.guru/design-patterns/bridge

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Facade

Structural Design Pattern

Expose a simple entry point to a complex subsystem.

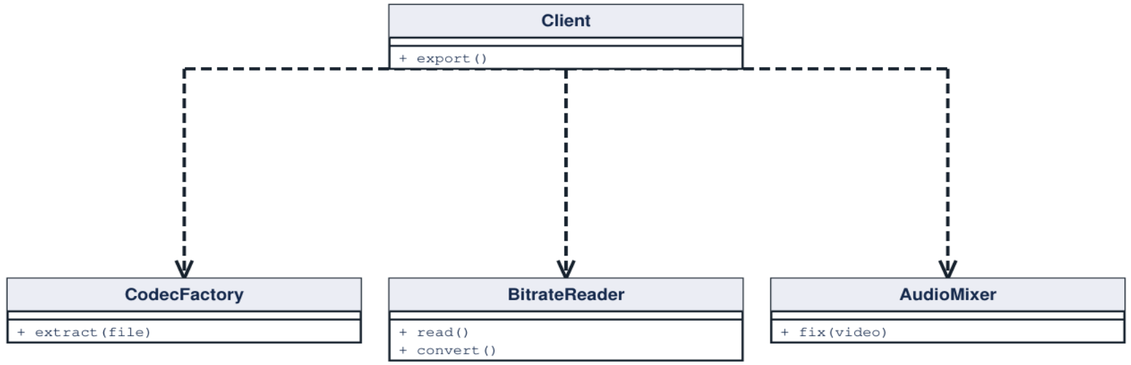
Problem

Every client coordinates codecs, bitrate conversion, and audio repair.

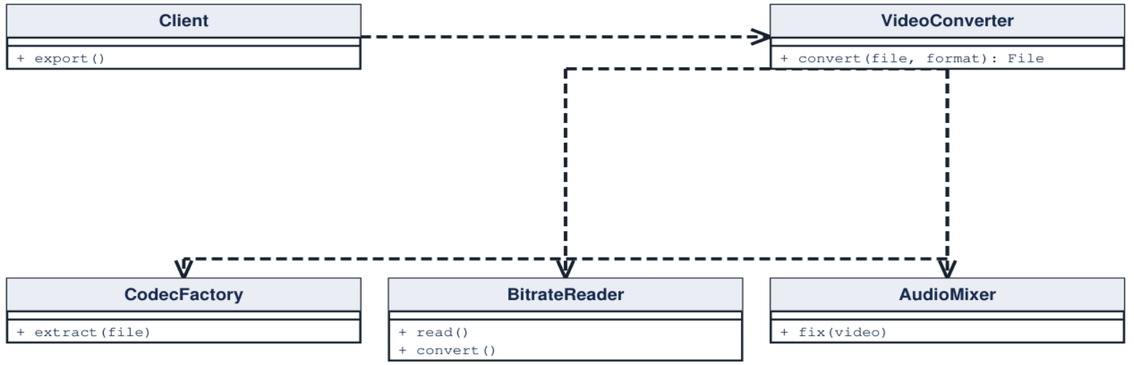
Solution

A facade orchestrates the subsystem behind one conversion call.

Before (direct coupling)



After (using Facade)



Client Code (Before)

Pseudocode

```
codec = CodecFactory.extract(file)
raw = BitrateReader.read(file, codec)
video = BitrateReader.convert(raw, mp4)
result = AudioMixer.fix(video)
```

Client Code (After)

Pseudocode

```
converter = new VideoConverter()
result = converter.convert(file, "mp4")
```

Key Implementation / Notable Module

Pseudocode

```
// VideoConverter.convert(file, format)
source = CodecFactory.extract(file)
raw = BitrateReader.read(file, source)
video = BitrateReader.convert(raw, codecFor(format))
return AudioMixer.fix(video)
```

Source: https://refactoring.guru/design-patterns/facade

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Proxy

Structural Design Pattern

Control access to a service through the same interface.

Problem

Repeated video downloads invoke the remote service again.

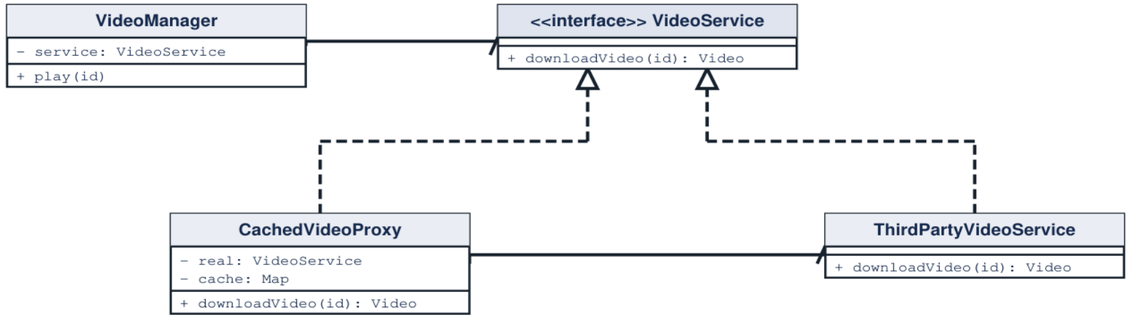
Solution

A proxy caches results and delegates only on a cache miss.

Before (direct coupling)



After (using Proxy)



Client Code (Before)

Pseudocode

```
service = new ThirdPartyVideoService()
a = service.downloadVideo(id)
b = service.downloadVideo(id)
// two remote requests
```

Client Code (After)

Pseudocode

```
service = new CachedVideoProxy(realService)
a = service.downloadVideo(id)
b = service.downloadVideo(id)
// second request uses the cache
```

Key Implementation / Notable Module

Pseudocode

```
// CachedVideoProxy.downloadVideo(id)
if not cache.contains(id):
    cache[id] = real.downloadVideo(id)
return cache[id]
```

Source: https://refactoring.guru/design-patterns/proxy

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Composite

Structural Design Pattern

Treat individual objects and recursive groups through one interface.

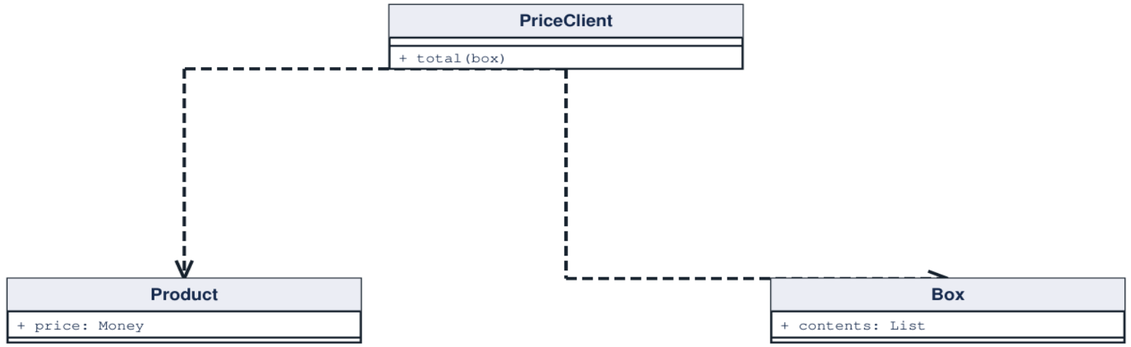
Problem

Pricing code distinguishes products from nested boxes repeatedly.

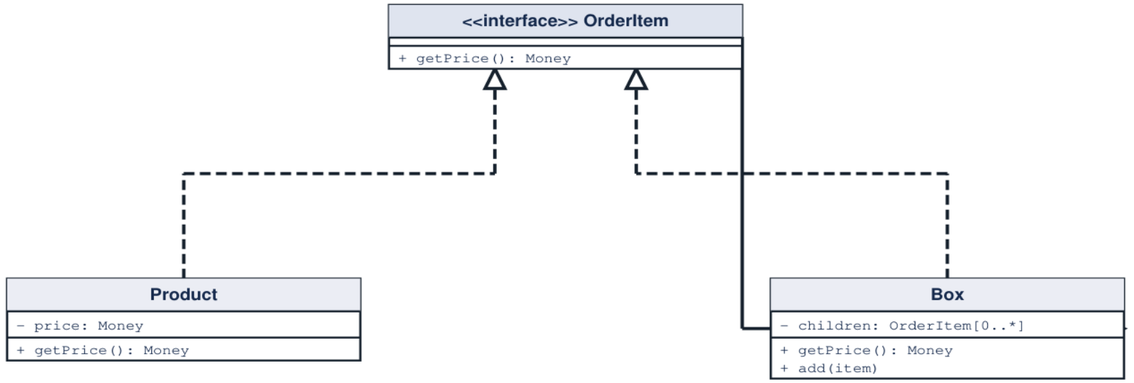
Solution

A box sums child prices; each child follows the same contract.

Before (direct coupling)



After (using Composite)



Client Code (Before)

Pseudocode

```
total = 0
for item in box.contents:
    if item is Product: total += item.price
    else: total += priceNestedBox(item)
```

Client Code (After)

Pseudocode

```
order = new Box([product, innerBox])
total = order.getPrice()
// leaf and group are both OrderItem
```

Key Implementation / Notable Module

Pseudocode

```
// Box.getPrice()
total = 0
for child in children:
    total += child.getPrice()
return total
```

Source: https://refactoring.guru/design-patterns/composite

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Template Method

Behavioral Design Pattern

Fix an algorithm’s order while subclasses implement selected steps.

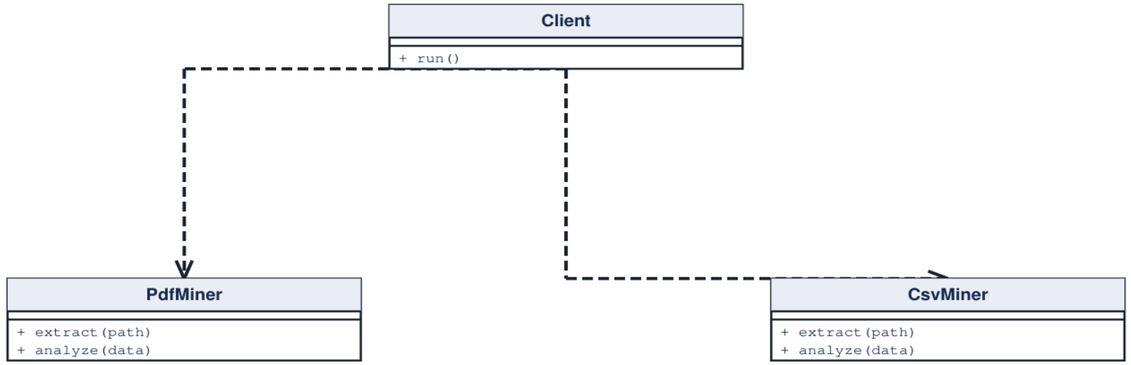
Problem

Format-specific miners duplicate the processing and reporting pipeline.

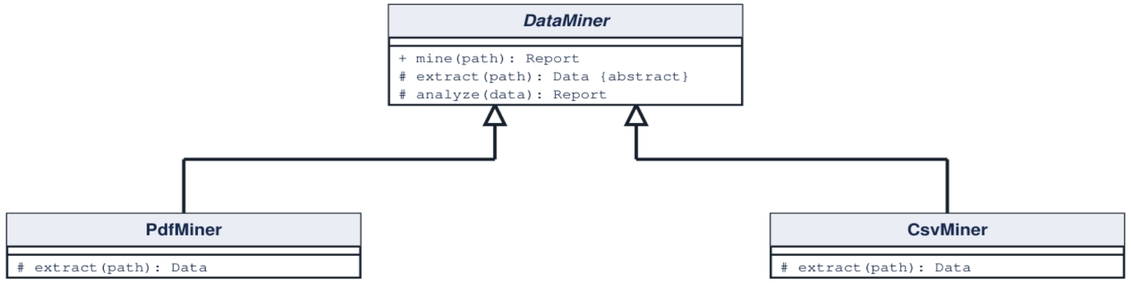
Solution

Move the sequence into a base method; override extraction only.

Before (direct coupling)



After (using Template Method)



Client Code (Before)

Pseudocode

```
if format == "pdf":
    data = pdf.extract(path)
else: data = csv.extract(path)
report = analyze(data)
```

Client Code (After)

Pseudocode

```
miner = new PdfMiner()
report = miner.mine(path)
// CsvMiner uses the same pipeline
```

Key Implementation / Notable Module

Pseudocode

```
// DataMiner.mine(path): fixed sequence
data = extract(path)
return analyze(data)
// PdfMiner.extract(path): return parsePdf(path)
```

Source: https://refactoring.guru/design-patterns/template-method

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Strategy

Behavioral Design Pattern

Make alternative algorithms interchangeable through a shared contract.

CASE STUDY: Navigation route algorithms

Problem

A navigator grows conditionals for road, walking, and transit routes.

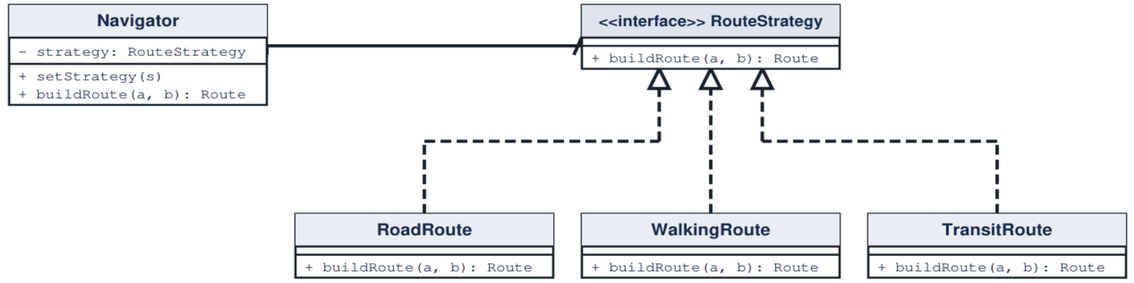
Solution

Delegate routing to a strategy selected by the client.

Before (direct coupling)



After (using Strategy)



Client Code (Before)

Pseudocode

```
if mode == "road": route = roadRoute(a, b)
elif mode == "walk": route = walkRoute(a, b)
else: route = transitRoute(a, b)
```

Client Code (After)

Pseudocode

```
navigator.setStrategy(new WalkingRoute())
route = navigator.buildRoute(a, b)
// choose another strategy at runtime
```

Key Implementation / Notable Module

Pseudocode

```
// Navigator.buildRoute(a, b)
return strategy.buildRoute(a, b)
// WalkingRoute builds pedestrian paths
```

Source: https://refactoring.guru/design-patterns/strategy

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

State

Behavioral Design Pattern

Delegate behavior to an object representing the current state.

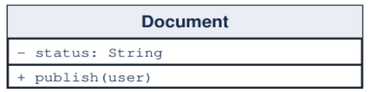
Problem

Publication rules accumulate in a switch over document status.

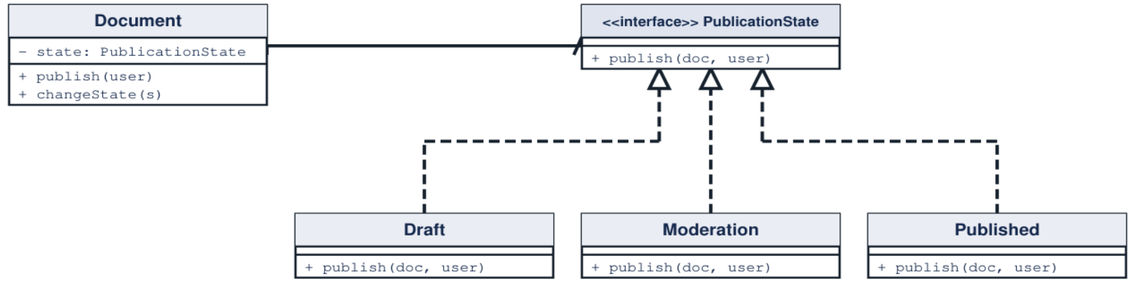
Solution

Each state handles publish and can transition the document.

Before (direct coupling)



After (using State)



Client Code (Before)

Pseudocode

```
if doc.status == "draft":
    doc.status = "moderation"
elif doc.status == "moderation" and user.isAdmin:
    doc.status = "published"
```

Client Code (After)

Pseudocode

```
doc = new Document(new Draft())
doc.publish(user)  // enters moderation
doc.publish(admin) // becomes published
```

Key Implementation / Notable Module

Pseudocode

```
// Moderation.publish(doc, user)
if user.isAdmin:
    doc.changeState(new Published())
// Published.publish(...) does nothing
```

Source: https://refactoring.guru/design-patterns/state

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Observer

Behavioral Design Pattern

Notify registered subscribers when a publisher emits an event.

CASE STUDY: Editor events notify logging and email

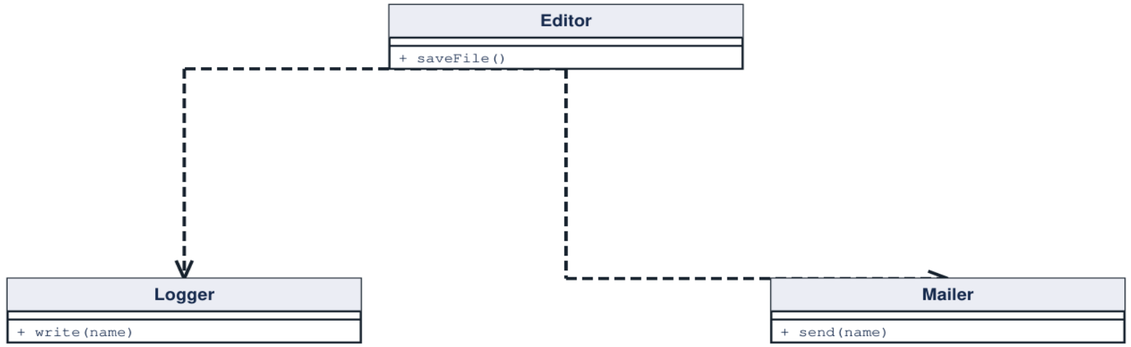
Problem

Editor logic directly invokes concrete logging and email services.

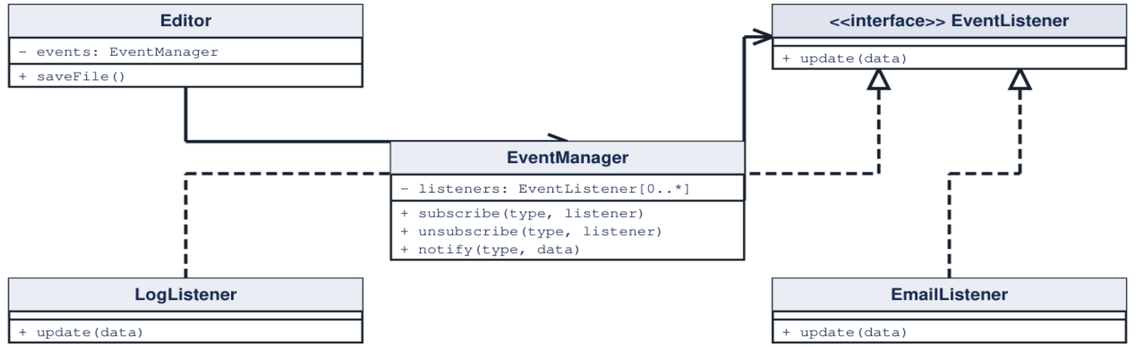
Solution

Publish an event; subscribers handle their own reactions.

Before (direct coupling)



After (using Observer)



Client Code (Before)

Pseudocode

```
editor.saveFile()
logger.write(file.name)
mailer.send(file.name)
// client must wire every reaction
```

Client Code (After)

Pseudocode

```
events.subscribe("save", new LogListener())
events.subscribe("save", new EmailListener())
editor.saveFile() // emits save event
```

Key Implementation / Notable Module

Pseudocode

```
// EventManager.notify(type, data)
for listener in listeners[type]:
    listener.update(data)
// Editor.saveFile(): write(); events.notify("save", name)
```

Source: https://refactoring.guru/design-patterns/observer

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency

Command

Behavioral Design Pattern

Represent a request as an object that can be invoked and undone.

CASE STUDY: Undoable text-editor actions

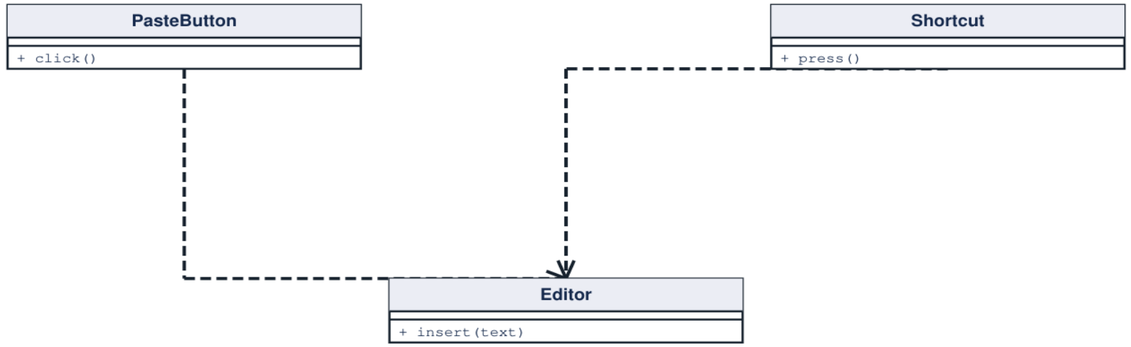
Problem

Buttons and shortcuts duplicate editing actions without shared undo.

Solution

Bind both triggers to a command; store successful edits in history.

Before (direct coupling)



After (using Command)



Client Code (Before)

Pseudocode

```
pasteButton.onClick(() => editor.insert(text))
shortcut.onPress(() => editor.insert(text))
// duplicated action; no undo record
```

Client Code (After)

Pseudocode

```
paste = new PasteCommand(editor, text)
pasteButton.bind(() => app.execute(paste))
shortcut.bind(() => app.execute(paste))
app.undo()
```

Key Implementation / Notable Module

Pseudocode

```
// PasteCommand.execute()
backup = editor.text; editor.insert(text)
return true  // app records command in history
// undo(): editor.text = backup
```

Source: https://refactoring.guru/design-patterns/command

UML: hollow triangle = inheritance / realization; dashed open arrow = dependency